

Improved Morphological Anti-Aliasing for Japanese Animation

Tanguy Cesaratto
OLM Digital, Inc.
Japan
cesaratto@olm.co.jp

Alexandre Derouet-Jourdan
OLM Digital, Inc.
Japan
alexandre.derouet-
jourdan@olm.co.jp

Marc Salvati
OLM Digital, Inc.
Japan
salvati.marc@olm.co.jp

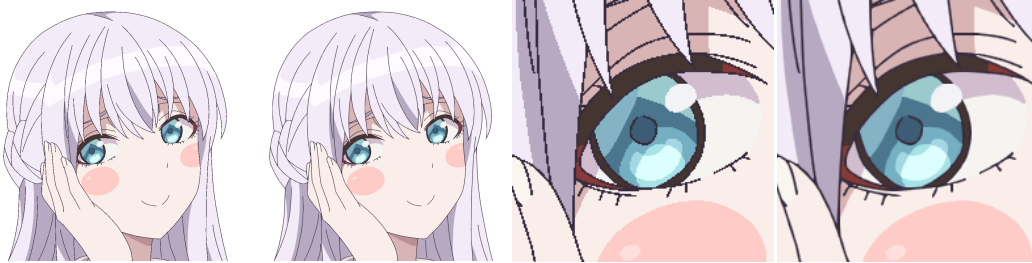


Figure 1: Original and anti-aliased. ©IIS-P/"Ponnomichi" Production Committee.

Abstract

Digitalization of 2D animation deeply modified artists' workflows and brought many challenges in that industry. While vector-based technologies have been adopted in many fields, Japanese animation still heavily relies on raster aliased drawings in their workflows. Thus post-process anti-aliasing solutions tailored towards traditional Japanese animation were developed based on Morphological Anti-Aliasing (MLAA). After a decade, MLAA has been refined and extended, especially in the context of 3D real-time rendering. In this paper, we present a revised solution which incorporates features from existing anti-aliasing techniques, tailored towards Japanese 2D animation.

Keywords

Japanese Animation, Cartoon, MLAA, Image Processing

ACM Reference Format:

Tanguy Cesaratto, Alexandre Derouet-Jourdan, and Marc Salvati. 2024. Improved Morphological Anti-Aliasing for Japanese Animation. In *SIGGRAPH Asia 2024 Technical Communications (SA Technical Communications '24)*, December 03–06, 2024, Tokyo, Japan. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3681758.3697990>

1 Introduction

Traditional Japanese animation pipelines roughly consist of the following steps: storyboarding, keyframe drawing, in-betweening, colorization, and compositing. Although artists are attached to pen and paper drawing, nowadays most pipelines use digital assets, at

least from the colorization step. In Japan, most productions use raster drawings, our animation studio included, and very few productions use vector drawing. Pixelated contour lines are used to enable efficient colorization of the raster regions. Pixelated drawings are also used at compositing time to create pixel exact masks. Thus in order to produce a smooth final image, a post-process anti-aliasing filter, such as Morphological Anti-Aliasing (MLAA [Reshetov 2009]), is applied (see Figure 2).

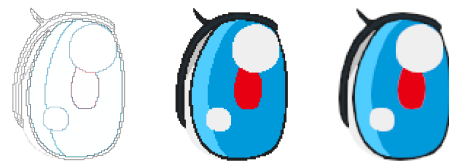


Figure 2: Contours, colorized, final result. ©B/T-CCTM.

MLAA [Reshetov 2009] removes jagged edges based on the rasterized shapes found in the image like shown in Figure 3. These rasterized shapes are reconstructed from the input aliased image, by recognizing certain patterns (L/Z/U shapes) made by edges. Pixels are finally blended based on the pixel coverage of these shapes.

At the time of the publication of MLAA, we had already developed an in-house anti-aliasing solution: OLM Smoother [Salvati et al. 2013]. The main difference with MLAA is its specialization for Japanese animation. OLM Smoother was published as a free tool and is used nowadays in many studios. A commercial anti-aliasing solution is also available: PSOFT [P SOFTHOUSE Co Ltd 2008]. Over time, several MLAA variations were proposed, applying the original algorithm to real-time rendering in the video game industry. While SMAA [Jimenez et al. 2012] is not tailored towards Japanese animation, it improves the quality of MLAA notably with the addition of diagonals handling. In this paper we explain how we improve OLM Smoother and address its limitations by considering more pixels patterns, adapting SMAA diagonals handling

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SA Technical Communications '24, December 03–06, 2024, Tokyo, Japan

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-1140-4/24/12

<https://doi.org/10.1145/3681758.3697990>

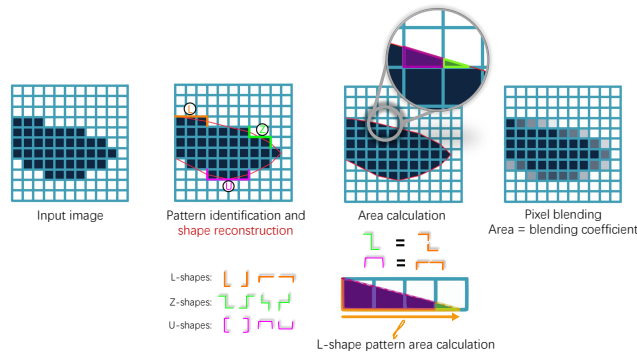


Figure 3: MLAA overview.

in the context of Japanese animation, adding control over gamma correction and smoothness parameters.

2 Pixels Patterns

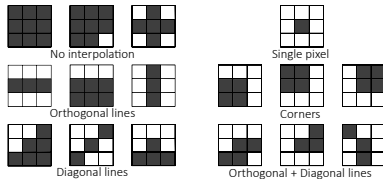


Figure 4: Pixels patterns recognized by our approach.

For each pixel in the image, we first find similar pixels in the neighborhood to determine if and how the shape will be reconstructed inside this pixel. We identify and manage the cases as listed in Figure 4 to search for the underlying shape. If two orthogonal and diagonal lines cross each other, we mix the result of both. Depending on the reconstructed shape within the pixel, we retrieve pairs of blending weights and colors to blend with, and finally mix the final color. This approach provides greater accuracy compared to a per-edge processing method (like in SMAA [Jimenez et al. 2012] for example) at the cost of performance (two treatments are required per edge instead of one).

2.1 Corners

After identifying a corner, we assign a blending weight to the innermost pixel proportional to the corner's size. Additionally, we also pay attention to the colors outside the corner to ensure that the innermost pixel of the corner remains unblended if it is surrounded by regions of different colors. The result is similar to what OLM Smoother [Salvati et al. 2013] does. SMAA has a parameter to modulate the corners rounding, which we set to 0 in order to achieve similar results. We can see in Figure 5 that our approach better preserves the corners.

2.2 Crossing Edges

Just as MLAA does, we look at the crossing edges at both ends of the edges line to determine the line's pattern and thus reconstruct the shape accordingly. However, the shape reconstruction performed



Figure 5: (From top to bottom, left to right) Original, OLM Smoother, SMAA, PSOFT, our approach.

in the original MLAA algorithm [Reshetov 2009] may not be true to the original shape due to insufficient crossing edges information. In fact, the reconstruction of the shape using MLAA's L/Z/U patterns is insufficient to handle areas with more than two different colors, producing an artifact we call color leak. As depicted in Figure 6, using only L/Z/U patterns causes the black stroke to leak into adjacent regions. Furthermore, determining the underlying shapes is not trivial, as the regions, for example, separate at different positions along the stroke.

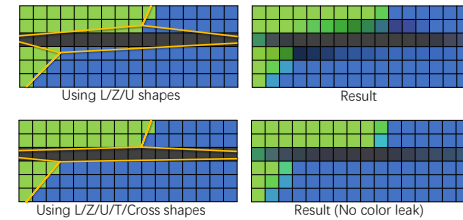


Figure 6: Comparison using additional patterns for the shape reconstruction.

This artifact is not really an issue when applying to 3D video games, since there is a lot of detail in the image, many tiny shapes are detected making the artifact barely visible. On the contrary, as explained in Section 1, for cel animation there may be long strokes and large filled regions that contrast with each other, which would produce noticeable undesired artifacts in the result image. This scenario arises when dealing with orthogonal lines, hence we direct our attention to them.

In our approach, we gather more information and specify more crossing edges patterns, thereby enabling the identification of T and cross shapes. We also fetch the extensions of crossing edges in order to handle single-pixel lines correctly like shown in Figure 7.

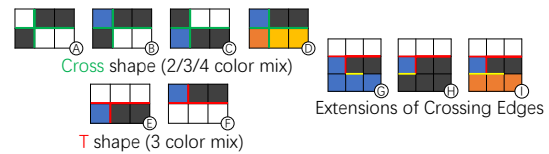


Figure 7: Crossing edges recognized in our approach.

2.2.1 Cross Shapes. Detecting cross shapes (see Figure 7 A-D) helps to accurately link disconnected lines. We use Z shapes for the shape reconstruction instead of MLAA's original approach which would recognize U shapes like shown in Figure 8. OLM Smoother

[Salvati et al. 2013] and PSOFT also connect these shapes in this way. SMAA approach does not always handle these cross shapes and may recognize a Z shape (see a production example in Figure 9).

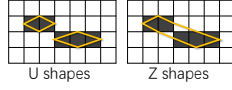


Figure 8: MLAA and our approach.



Figure 9: Original, OLM Smoother, SMAA, PSOFT and our approach. ©Nabeshiki/ESE/ I PARRY EVERYTHING.

2.2.2 T Shapes. T-shapes (see Figure 7 E-F) indicates that there are three different colors at the line end. We can first consider reconstructing the shape going through the intersection of these three colors. In such a case, colors will not leak (like shown in Figure 6), however pixels near the T-shapes will not look smooth. To address this issue, we first identify the pattern (U/Z shape) if there is one. Next, we analyze in detail and find a single pixel segment we want to interpolate (see Figure 10). In our original approach, OLM Smoother had color leak because it only recognized U/Z shapes (notice on Figure 10 that the color blending is not always correct). SMAA approach results in color leak at line extremities. While PSOFT [P SOFTHOUSE Co Ltd 2008] result does not suffer from this artifact, there are some single pixel segments that are not interpolated, resulting in high contrast neighbor pixels.



Figure 10: Original, OLM Smoother, SMAA, PSOFT and our approach.

2.2.3 U-pattern adjustment. SMAA adjusts the reconstructed shape for U-patterns by bending the shape towards the line. We adjust it in a very similar way by first calculating the interpolation weight for a normal U-pattern and apply the following $w = 2w^2$ with w the interpolation weight and $w \in [0, 0.5]$. This allows to have a rounded U-pattern as shown in Figure 11.

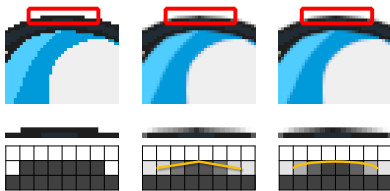


Figure 11: Original, naive, with adjustment. ©B/T-CCTM.

2.2.4 Single Pixel Lines. Knowing the extensions of crossing edges (see Figure 7 G-I), not only can we know whether the current line we are focusing on is a single pixel line, but also its neighbor one. In fact, if we have two single pixel lines side by side (I in Figure 7), we need to tune the formula to achieve coherent blending results (see Figure 12).

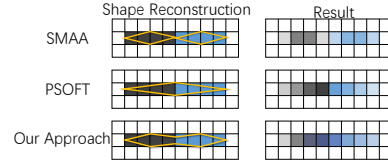


Figure 12: Comparison of single-pixel lines handling.

2.3 Diagonal Lines

OLM Smoother originally did not consider diagonal lines but recognized a sequence of orthogonal lines leading to blocky anti-aliased diagonal lines. SMAA [Jimenez et al. 2012] improved their look by recognizing diagonal lines in the shape reconstruction, technique which we incorporated in our revised version. Still, we improve the quality of the shape reconstruction at the cost of a deeper edges analysis as shown in Figure 13. We also consider that diagonal lines consist of two colors implying that we check for diagonal edges along the diagonal line. Finally, the pixels patterns mentioned in Section 2 allow us to accurately reconstruct the shape when having diagonal and orthogonal lines in the same pixel. SMAA approach sometimes results in a faded line and irregular stroke width.

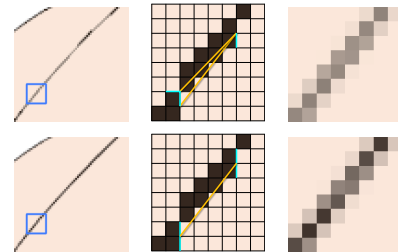


Figure 13: SMAA (top) and our approach (bottom). ©Nabeshiki/ESE/ I PARRY EVERYTHING.

3 Gamma Correction

In order to differentiate and blend colors accurately based on their intensity, we linearize them by applying gamma correction on input and output images (which are in sRGB color space). MLAA variations for cel animation, such as OLM Smoother [Salvati et al. 2013] and PSOFT [P SOFTHOUSE Co Ltd 2008], yet do not include such correction in their algorithms leading sometimes to darker and bluer results when using sRGB color space (see the comparison in Figure 14).

Although users gave us overall positive feedback, they also reported that black thin (1 pixel wide) strokes often mixed too much with bright regions resulting in somewhat discontinuous black thin



Figure 14: Original, using sRGB, using linear. ©B/T-CCTM.

strokes (see Figure 15). We extended the feature to enable an additional and custom gamma compression and expansion, prior to and following the colors blending. This extended gamma correction uses a simple power-law expression: $V_{out} = V_{in}^{\gamma}$. By adding a user-defined gamma value along with a list of custom colors to which this gamma correction is applied, users were able to fine-tune black thin lines in a tailored manner (see Figure 15).

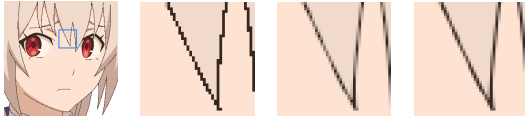


Figure 15: Original, without and with extended gamma correction. ©Nabeshiki/ESE/ I PARRY EVERYTHING.

4 Parametrizable Smoothness

We implemented a feature allowing users to control the shape reconstruction from the original aliased shape to the presumed true shape, corresponding to the level of smoothing applied to the aliased image. We manage orthogonal lines separately from other cases, for which we simply scale the blending weight by the parameter's value. For orthogonal lines, as described in MLAA [Reshetov 2009], most patterns found (L/Z/U shapes) can essentially be represented using one or multiple L shape(s), used to calculate the blending weights (see the L-shape pattern area calculation in Figure 3). We introduce the parameter in the calculation by scaling down the L shape by the parameter's value: $l = l \times s$ with the length of the L letter l , the smoothness parameter $s \in [0, 1]$.

In addition, we introduced another parameter to add extra smoothing to the final image. The process is similar to the one described above, except that we modify the shape reconstruction from the presumed true shape to an extended one, by scaling up the L shape using the edges line length like shown in Figure 16 and with a production example in Figure 17. PSOFT's solution also allows user to control smoothness and give similar results.

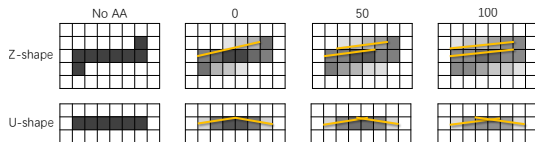


Figure 16: Shape reconstruction with extra smoothness.



Figure 17: Original, extra smooth [0, 100]. ©IIS-P/"Ponnomicchi" Production Committee.

5 Conclusions and Future Works

Artists in our studio evaluated the different anti-aliasing solutions discussed in the paper and preferred our revised version. They were fully satisfied of the corrections and improvements added to the previous version and particularly liked the gamma correction and smoothness parameters which enable them to fine-tune the width of the lines. Our solution has been integrated into their production pipeline and is in use for upcoming productions. Our software was implemented as a C++ library with the aim to use it in various softwares, especially our in-house 2D drawing software under development. Hence, the library supports multiple image data formats and is able to anti-alias portions of an image. We made tests using graphic design softwares and concluded that user experience is sufficiently good enough for interactive use, with a performance within a [10 - 15] ms range using 1080p images, which is similar to other CPU-based anti-aliasing solutions. Nonetheless, we may further improve its real-time capabilities by developing a GPU version. We also plan to improve our shape reconstruction considering the color leak problem mentioned in Section 2.2.2. Our current approach may result in too hard shapes, we could mitigate that problem by adding a controlled color leak. Concerning the gamma correction's extension in Section 3, we aim to test an optional color space based on human's color perception (similar to OSA-UCS) which could overcome the limitations met using linear colors.

References

- Jorge Jimenez, Jose I. Echevarria, Tiago Sousa, and Diego Gutierrez. 2012. SMAA: Enhanced Morphological Antialiasing. *Computer Graphics Forum (Proc. EUROGRAPHICS 2012)* 31, 2 (2012).
- P SOFTHOUSE Co Ltd. 2008. PSOFT anti-aliasing. <https://www.psoft.co.jp/jp/product/anti-aliasing/ae/1.1.0>.
- Alexander Reshetov. 2009. Morphological Antialiasing. In *High-Performance Graphics*. The Eurographics Association, 109–116.
- Marc Salvati, Yosuke Katsura, and Ken Anjyo. 2013. MorphoLogical Anti-Aliasing in Production (*DigiPro2013*).